Step 1 — A Script That Sends One Message and Prints the Reply — Hints

> Back to this step · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've really tried. Each hint starts with the plain idea, then shows the shape of the real code.

- Hint 1 — The pieces, and where each one lives
- Hint 2 — The plan, and almost the whole thing

> Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — The pieces, and where each one lives

This step proves one thing: your code can load its settings and get a sensible reply from the model. No loop, no memory, no streaming yet.

You need four pieces, each in its final folder:

- `src/exceptions.py`, `src/config.py` and `src/utils/logger.py` — Doc01's Build Task files. Copy them in; the only change is the import line in `config.py`: `from src.exceptions import MissingConfigError`.
- `src/models/llm.py` — builds the OpenAI client and sends a message.
- `main.py` — the file you run.
- An empty `__init__.py` in `models/` and `utils/`, so Python treats them as packages.

Things you'll use (all from Doc04):

- `OpenAI(api_key=...)` — builds the client.
- `client.chat.completions.create(model=..., messages=[...])` — the call.
- The reply text is at `response.choices[0].message.content`.

`models/llm.py` needs three functions and one constant:

```
MODEL = "gpt-4o-mini"

def create_client(config: Config) -> OpenAI: ...
def check_api_key_at_startup(client: OpenAI) -> bool: ...
def send_message(client: OpenAI, history: list[dict], user_input: str) -> str:
    ...
```

- `create_client()` passes `config.openai_api_key` in by hand, so the key always comes from the one place that checked it.
- `send_message()` is Doc04's Build Task function: append the user turn to `history`, call the model, append the reply, return it. For one message, `history` just starts as `[]`.
- `check_api_key_at_startup()` sends a tiny request (`max_tokens=1`) and returns `False` if it raises `openai.AuthenticationError`. It costs almost nothing, and a bad key is found before the user types anything.

In `main.py`, order matters: `load_config()` first (it loads `.env`), *then* `get_logger(__name__)` (it reads `LOG_LEVEL`), then the client and the key check.

Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

```
main():
    config = load_config()
    client = create_client(config)
    history = []
    reply = send_message(client, history, "What's 2+2?")
    print(reply)

run it from the repo root:  python main.py
```

If you get `ModuleNotFoundError: No module named 'src'`, you're not running it from the repo folder, next to `src/` (Setup, part 5).

The same plan, with both failures handled cleanly:

```
main():
    try:
        config = load_config()
    except MissingConfigError as e:
        print a "Setup problem" line, then sys.exit(1)

    logger = get_logger(__name__)
    client = create_client(config)
    if not check_api_key_at_startup(client):
        logger.error(...), print a "Login failed" line, sys.exit(1)

    logger.info("Sending a test message to the model")
    reply = send_message(client, [], "What's 2+2?")
    print(reply)
```

Test both failures on purpose:

- Delete the key from `.env`: you should see one "Setup problem" line.
- Set it to `sk-fake-key-123`: you should see one "Login failed" line.

Neither should show a traceback.

Full solution: Show me the solution